

Resolución neuro-simbólica del benchmark ARC-AGI utilizando modelos de lenguaje ligeros y representación simbólica eficiente

Saymon Nicho

Asesores: Dr. César Beltrán, Dr. Edwin Villanueva

PUCP

4 de diciembre de 2025

Contenido

1 Introducción

2 Metodología

3 Resultados

4 Conclusiones

Introducción

Introducción

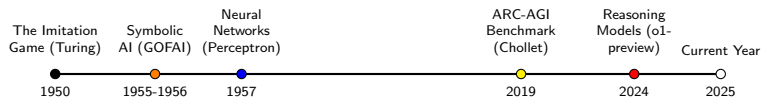


Figura 1: Historia de la IA (elaboración propia)

ARC-AGI

Propuesto por **Francois Chollet** en 2019 como un benchmark para evaluar *human-like intelligence* en modelos de IA ^[1].

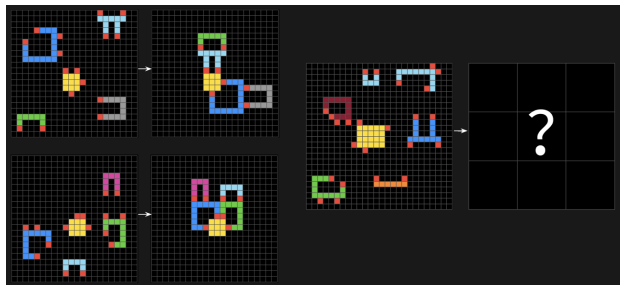


Figura 2: Benchmark ARC-AGI ^[1]

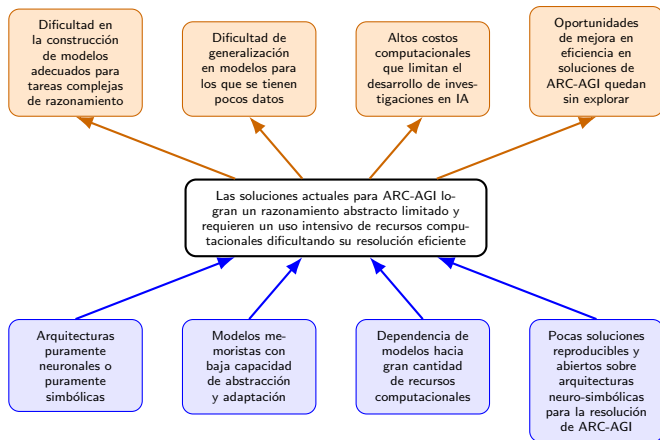


Figura 3: Árbol de Problemas

Problema Seleccionado

La problemática principal es la **difícultad actual que presentan los modelos de IA** para demostrar capacidades de **razonamiento y abstracción**, así como **generalización** efectiva a partir de pocos ejemplos.

En benchmarks como ARC-AGI, las arquitecturas actuales basadas en LLMs tienen **limitaciones significativas en términos de eficiencia**.

El Dataset ARC

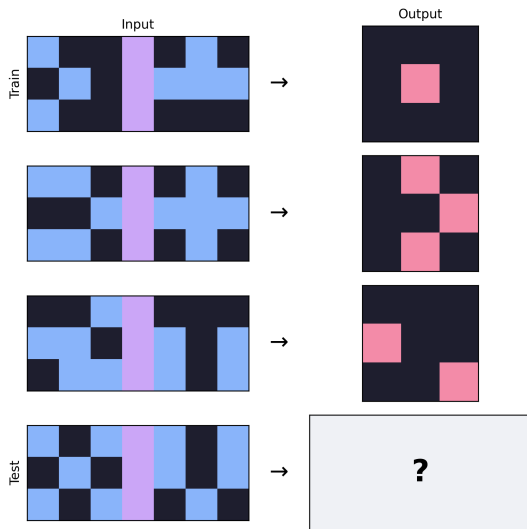


Figura 4: Tarea 0520fde7 de ARC [1]

Métodología

- **Deep Learning Guided Program Synthesis:** combina aprendizaje neuronal con reglas explícitas en un DSL
- **Test-Time Fine-Tuning (TTFT):** adapta el modelo en inferencia para mejorar precisión
- **Data Augmentation:** aumenta diversidad de ejemplos en un dataset limitado como ARC
- **Ensamble Inductivo–Transductivo:** combina razonamiento simbólico y predicción directa

Datasets Seleccionados

- **ARC-AGI**: dataset original, 400 tareas de train y 400 de evaluación
- **ConceptARC**: 160 tareas agrupadas por conceptos (simetría, conteo, orientación, etc.)
- **Re-ARC**: tareas similares a las del dataset original pero con generadores de hasta 10k ejemplos por tarea
- **ARC-Heavy**: 200k tareas sintéticas para escalar entrenamiento de síntesis de programas

Pipeline Base - Entrenamiento

- **Módulo Inductivo:** sintetiza programas con el DSL
- **Módulo Transductivo:** predice salidas directamente y usa TTFT

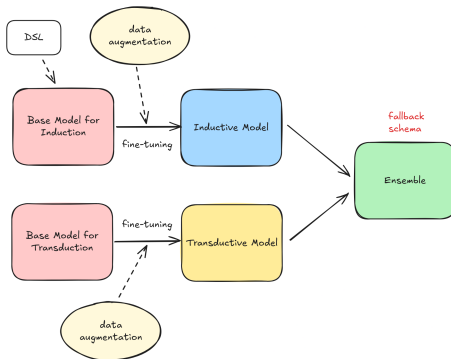


Figura 5: Esquema de entrenamiento

Entrenamiento del Módulo Transductivo

- **Modelos:** Qwen2.5-3B-Instruct y Llama-3.2-3B-Instruct
- **Fase offline:** fine-tuning supervisado en Re-ARC
- **Test-Time Fine-Tuning:** ajuste por tarea usando data augmentation
- **Optimización:** LoRA ($r=64$), cuantización 4-bit, tokenizer reducido

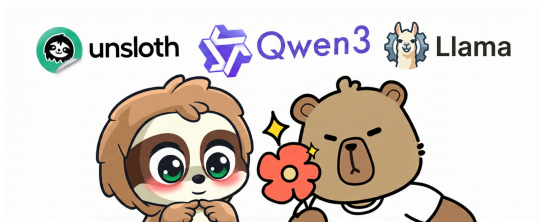


Figura 6: Biblioteca Unsloth (generado con Nano Banana Pro)

Síntesis de Programas en el Módulo Inductivo

- **Modelo:** Qwen2.5-Coder-1.5B
- **Datasets:** ARC-Heavy (diversidad) + ConceptARC (conceptos)
- **Objetivo:** generar programas simbólicos correctos en DSL

Pipeline Base - Inferencia

- **Ensamble:** primero se intenta la solución del módulo inductivo; si falla, se usa el transductivo.

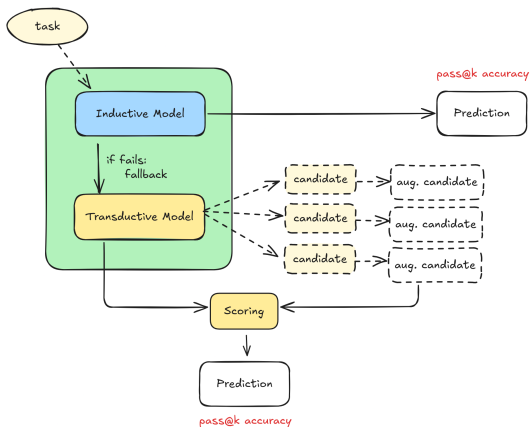


Figura 7: Esquema de inferencia

Resultados

La métrica utilizada fue $pass@2$, definida como:

$$pass@2 = \frac{1}{N} \sum_{i=1}^N 1\{\exists j \in \{1, 2\} : \hat{y}_{i,j} = y_i\}$$

Es decir, una tarea se considera resuelta si al menos una de las dos predicciones generadas (ya sea del mismo módulo o de módulos distintos) coincide exactamente con la salida correcta.

Evaluación de los Modelos

Modelo	Vanilla	Sin TTFT	Con TTFT
Qwen2.5-3B-Instruct (baseline)	4.0 %	—	—
Llama-3.2-3B-Instruct-ReARC-merged	—	20.0 %	30.5 %
Qwen2.5-3B-Instruct-ReARC-merged	—	22.5 %	34.0 %

Cuadro 1: Resultados de evaluación del módulo transductivo (pass@2)

Modelo	Vanilla	Con PS
Qwen2.5-Coder-1.5B (baseline)	2.5 %	—
Qwen2.5-Coder-1.5B-Finetuned	—	26.5 %

Cuadro 2: Resultados de evaluación del módulo inductivo (pass@2)

Configuración	pass@2 Accuracy
Módulo Transductivo (mejor)	34.0 %
Módulo Inductivo	26.5 %
Ensamble de Ambos Módulos	41.0 %

Cuadro 3: Resultados de evaluación del ensamble (pass@2)

⁰Todos los resultados corresponden al promedio de 2 ejecuciones independientes.

Comparación con el Estado del Arte

Método / Equipo	Conjunto	pass@2 Accuracy
<i>ARChitects</i> (2024 Winner)	Privado (100)	53.5 %
<i>Guillermo Barbadillo</i>	Privado (100)	40.0 %
<i>alijs</i>	Privado (100)	40.0 %
<i>William Wu</i>	Privado (100)	37.0 %
<i>PoohAI</i>	Privado (100)	37.0 %
Módulo Transductivo (mejor)	Público (100)	34.0 %
Módulo Inductivo	Público (100)	26.5 %
Ensamble propuesto	Público (100)	41.0 %

Cuadro 4: Desempeño del ensamble propuesto frente al SOTA (pass@2).

⁰Todos los resultados corresponden al promedio de 2 ejecuciones independientes.

Todas las pruebas se realizaron en una GPU NVIDIA RTX A6000 con 48 GB de memoria y con CUDA 12.8.

Método	GPU	Memoria	Tiempo por Tarea
Este trabajo (Transductivo)	RTX A6000	8-10 GB	2-5 min
Este trabajo (Inductivo)	RTX A6000	12 GB	4-8 min
Estado del Arte (P100)	P100	16 GB	20-60 s (estimado)

Cuadro 5: Comparación de recursos entre el proyecto y el SOTA

Conclusiones

Conclusiones

- Se implementó una arquitectura neuro-simbólica modular y eficiente.
- TTFT y síntesis de programas demostraron mejoras substanciales.
- El ensamble logró **41 %**, acercándose al SOTA con modelos pequeños.
- Se liberó código y documentación completa para reproducibilidad.



- Explorar más modelos y configuraciones de hiperparámetros.
- Analizar qué tipos de tareas beneficia cada módulo del pipeline.
- Evaluar en ARC-AGI v2.
- Integrar técnicas simbólicas y neuronales en un solo modelo unificado.

Referencias

- [1]. François Chollet.
On the measure of intelligence.
arXiv preprint arXiv:1911.01547, 2019.